ChatGPT

A Token-Efficient Workflow for Building Full Production Applications with Codex and Claude Code

Executive conclusion

The most efficient default workflow is **not** “build the whole backend, then build the whole frontend”, and it is usually **not** “have separate frontend and backend agents working independently”.

My recommendation is:

> **Do a short horizontal design/foundation pass first, then build the application as small end-to-end vertical slices, with one bounded slice per fresh implementation session. Treat Git plus a small set of repository Markdown files as the agent's durable memory, rather than relying on a giant continuous chat.**

In practice:

IDEA → product spec → architecture + contracts → repository control files → minimal project skeleton → vertical slice 1 → fresh session → vertical slice 2 → fresh session → … → integration/hardening → independent review → staging → production → post-deploy verification

The key distinction is that you **design the overall system horizontally but implement it vertically**.

This conclusion lines up closely with the current guidance from both vendors. OpenAI's August 2026 Codex workflow recommends breaking builds into individual phase files containing purpose, context, acceptance criteria and boundaries, specifically saying this reduces agent drift and creates a human-gated development history. It recommends that phases be small enough to implement and verify independently. [1] Anthropic recommends separating exploration/planning from implementation, writing self-contained specifications for larger features, and explicitly starting a **fresh session once the specification is complete**. [2]

The reason context discipline matters is straightforward: Anthropic states that token cost grows with context size and warns that stale context wastes tokens on subsequent turns. OpenAI similarly warns that filling the main thread with exploration output, logs and intermediate material causes “context pollution” and can make the session less reliable. [3]

The recommended pattern at a glance

Decision	Recommendation
Build entire frontend separately from backend?	**No, not by default.**

Decision	Recommendation
Build entire backend first?	**No, except for genuinely API-first/data-first systems.**
Build feature-by-feature end to end?	**Yes. This should be the normal implementation unit.**
Design architecture before coding?	**Yes, but only enough architecture to establish boundaries and contracts.**
Define database/API contracts early?	**Yes, particularly contracts needed by the first few slices. Do not spec every hypothetical endpoint.**
One long agent conversation for the whole app?	**No. Fresh session for each meaningful slice.**
Resume sessions?	**Only while continuing the same unfinished task.**
Put everything in `AGENTS.md` / `CLAUDE.md`?	**No. Put only durable, universally relevant rules there.**
Where should project state live?	**Version-controlled specs, task files, contracts, ADRs and `PROGRESS.md`.**
Strongest model for every task?	**No. Escalate model/reasoning only when task ambiguity or difficulty justifies it.**
Parallel agents?	**Sparingly. Prefer read-only investigation/review; use worktrees for independent write work.**

The remainder of this report turns that into an operational workflow.

Why vertical slices beat frontend-first or backend-first

The winning structure is horizontal design, vertical implementation

There are really two different questions:

1. **How should you understand/design the system?**
2. **How should you implement it?**

For design, you should briefly think horizontally across the whole product:

```
Users / workflows
       ↓
Domain model
       ↓
Architecture boundaries
       ↓
Database model
       ↓
API contracts
       ↓
Frontend architecture
```

```
       ↓
Infrastructure / deployment
```

For implementation, reverse the perspective:

```
Feature A
DB → domain logic → API → UI → tests → done

Feature B
DB → domain logic → API → UI → tests → done

Feature C
DB → domain logic → API → UI → tests → done
```

OpenAI's current Codex workflow guidance strongly supports small phases with explicit acceptance criteria, boundaries and relevant context, and recommends observable verification and dependency ordering rather than executing a huge build plan in one undifferentiated run. [1] Anthropic likewise recommends an explore → plan → implement → verify loop rather than asking the agent to immediately code a large ambiguous request. [4]

My recommendation of **vertical slices** is a synthesis of those principles rather than a Codex- or Claude-specific mandate.

Why not build the frontend separately?

Imagine a CRM application containing:

```
Authentication
Companies
Contacts
Opportunities
Pipeline
Activities
Dashboard
Notifications
Billing
```

A frontend-first workflow might produce thirty screens against mocked APIs before the real backend exists.

That causes three agent-efficiency problems.

First, the eventual backend agent has to rediscover all the assumptions embedded in the UI. Second, integration becomes a separate, large task with broad repository context. Third, interface mismatches are discovered late, generating further modification passes across already-working code.

Instead, build:

```
Slice 01: User can sign in
Slice 02: User can create and view a company
Slice 03: User can add a contact to a company
Slice 04: User can create an opportunity
Slice 05: User can change opportunity stage
...
```

Each slice touches only the subset of schema, backend, API, UI and tests required for that behaviour.

That fits OpenAI's advice to tell Codex exactly which repository sources to read, reference existing plans rather than repeating them, define immediate scope and explicit non-goals, and perform a read-only preflight before implementation. [5] It also fits Anthropic's recommendation to make specifications self-contained by naming relevant files/interfaces, explicitly stating what is out of scope, and ending with end-to-end verification. [6]

Why not build the entire backend first?

Backend-first is useful when the backend **is effectively the product contract**: for example, a public API, a data platform, an integration service, a protocol implementation or a system where multiple independent clients will consume an externally stable API.

For most SaaS/web/mobile applications, however, I would make the backend first **inside each slice**, not for the entire application:

```
For Feature N:

acceptance criteria
        ↓
schema/migration if needed
        ↓
domain/backend logic
        ↓
API contract
        ↓
frontend against real API
        ↓
integration/E2E test
```

That gives the UI a real contract immediately while avoiding months of speculative backend endpoints that later need adapting.

A slice does not need to be large. Good slices are often something like:

> “An authenticated user can create a company containing a name and website, and can see it immediately in the companies list.”

Bad slice:

> “Build companies.”

The second instruction can expand into database design, search, pagination, permissions, CRUD, imports, tags, filters, analytics and other work that was never explicitly requested.

How small should a slice be?

Use this practical test:

> **Could a new agent session understand the slice from one task file plus roughly a handful of relevant implementation files and complete it with an observable test?**

If not, split it.

A task should ideally have **one user-visible outcome, one coherent code change, one verification path and one commit/PR**. This is consistent with OpenAI's recommendation that build phases contain purpose, context, acceptance criteria and boundaries and that phase-specific context should contain only material information that might otherwise cause later agents to repeat discovery or make inconsistent decisions. [1]

For example:

```
TOO BIG

TASK-010 Build team management

BETTER

TASK-010 Display organisation members
TASK-011 Invite a member by email
TASK-012 Accept an invitation
TASK-013 Change member role
TASK-014 Remove a member
```

This decomposition is one of the largest practical token savings you can make because the agent stops repeatedly reasoning about features that are not part of its current assignment.

The repository should become the agent's long-term memory

The single most important operational rule is:

> **Chats are disposable execution environments. The repository is the memory system.**

Claude can resume conversations and Codex can compact or resume work, but those mechanisms should be considered conveniences for an **unfinished task**, not your application project's canonical memory. Anthropic explicitly recommends clearing context between unrelated tasks because stale context wastes tokens, while Claude's resumable sessions are intended for a continuing workstream. [7] Codex offers `/compact`, which replaces earlier turns with a concise summary while preserving important details, making it useful when one legitimately long task needs to continue. [8]

Use this minimal documentation structure

I recommend this structure:

```
/
├── AGENTS.md
├── CLAUDE.md
├── PRODUCT.md
├── ARCHITECTURE.md
├── PLAN.md
├── PROGRESS.md
│
├── docs/
│   ├── adr/
│   │   ├── 0001-stack.md
│   │   ├── 0002-auth.md
│   │   └── ...
│   ├── contracts/
│   │   ├── openapi.yaml
│   │   └── ...
│   └── CODEMAP.md
│
├── tasks/
│   ├── 001-project-foundation.md
│   ├── 002-authentication.md
│   ├── 003-create-company.md
│   └── ...
│
└── src/...
```

OpenAI's current Codex cookbook proposes essentially the same separation of concerns through `AGENTS.md`, goals/plans, per-build phase files, phase-specific context and a build log. Importantly, it advises against duplicating ownership between documents. [9]

The names above are intentionally simpler than OpenAI's example harness. What matters is **single ownership of information**.

`AGENTS.md`: the durable operating constitution

`AGENTS.md` should not explain the whole application. It should tell an agent **how work must be performed in this repository**.

Codex discovers `AGENTS.md` when starting a run/session and can layer more specific instruction files as it descends from the repository root into subdirectories. [10]

I would make the root file something like:

```
# Repository working rules
```

```
## Sources of truth

- PRODUCT.md owns product requirements and non-goals.
- ARCHITECTURE.md owns system boundaries and high-level architecture.
- docs/adr/ owns accepted architectural decisions.
- docs/contracts/ owns external and cross-layer contracts.
- PLAN.md owns ordered feature/slice sequencing.
- tasks/<id>.md owns scope and acceptance criteria for the current task.
- PROGRESS.md owns observed project status and verification evidence.

Do not duplicate these sources into other documents.

## Before changing code

1. Read the current task file.
2. Read PROGRESS.md.
3. Read only the ADRs/contracts referenced by the task.
4. Inspect the relevant implementation and tests.
5. Check git status/diff.
6. Perform a read-only preflight before editing.

Do not perform a repository-wide exploration unless required to resolve an
unknown.

## Change discipline

- Implement only the current task.
- Do not begin the next task.
- Do not refactor unrelated working code.
- Do not rename/reformat unrelated files.
- Do not upgrade dependencies unless explicitly required.
- Preserve existing public contracts unless the task explicitly changes them.
- Follow existing patterns before introducing new abstractions.
- Prefer the smallest complete implementation satisfying acceptance criteria.

## Verification

- Run the narrowest relevant tests during development.
- Add regression tests for changed behaviour.
- Run all verification required by the task before completion.
- Never claim success without observed command/test evidence.

## Completion

Before stopping:

1. Inspect the complete git diff.
2. Remove accidental/out-of-scope changes.
3. Update PROGRESS.md with facts only.
4. Report changed files, tests run, results and unresolved issues.
5. Stop. Do not start another task.
```

This file has high leverage precisely because it is short.

Anthropic makes the same point for `CLAUDE.md`: it is loaded at the beginning of conversations, should contain broadly applicable build commands/conventions/workflow rules, and should remain concise; occasional domain-specific procedures belong in on-demand skills instead because oversized persistent instructions consume context and receive worse adherence. [11]

Do not maintain separate duplicated `AGENTS.md` and `CLAUDE.md`

Claude Code currently reads `CLAUDE.md`, not `AGENTS.md`, but Anthropic explicitly documents importing an existing `AGENTS.md` from `CLAUDE.md`. [12]

Therefore use:

```
@AGENTS.md

# Claude-specific instructions

When compacting, preserve:
- current task ID
- files changed
- tests run and their results
- unresolved blockers
```

This gives you **one canonical agent constitution** instead of maintaining two copies that inevitably diverge.

Anthropic documents custom compaction instructions for exactly this purpose. [13]

`PRODUCT.md`: what you are building

Keep:

```
Problem
Target users
Jobs/workflows
MVP
Explicit non-goals
Functional requirements
Business rules
Critical edge cases
Definition of production-ready
```

Do **not** put:

```
implementation progress
session logs
```

```
individual task instructions
code snippets that already live in source
```

A future agent should be able to understand *why the application exists* without reading implementation history.

`ARCHITECTURE.md`: stable map, not encyclopaedia

It should answer:

```
What are the major components?
What owns each responsibility?
What calls what?
Where is state stored?
Where are authentication/authorisation enforced?
What are deployment boundaries?
What invariants must remain true?
```

Aim for diagrams/tables and compact descriptions.

Do not produce a 100-page “architecture bible”. Every automatically or routinely loaded document becomes a potential context tax.

ADRs: preserve expensive decisions

For any decision expensive enough that another agent might reconsider it, make a short architecture decision record:

```
# ADR-0003: Application authentication

Status: Accepted

## Context
...

## Decision
...

## Consequences
...

## Do not revisit unless
...
```

This is much cheaper than having five future agents independently debate the same authentication architecture.

OpenAI specifically recommends carrying forward material information when its absence could cause a future phase to repeat discovery, change an interface/dependency/invariant or make a different implementation decision. [14]

`CODEMAP.md`: your antidote to repeated repository exploration

Do **not** create a file-by-file catalogue.

Create a stable, on-demand map:

```
# Code map

## Web application
apps/web/
Entry: apps/web/src/app/...
Owns pages and browser interaction.

## API
apps/api/
Entry: apps/api/src/server.ts
Routes: apps/api/src/routes/
Domain services: packages/domain/

## Persistence
packages/db/
Schema: packages/db/schema/
Repositories: packages/db/src/repositories/

## Shared API types
packages/contracts/

## Authentication
packages/auth/

## Tests
Unit: ...
Integration: ...
E2E: ...
```

Claude's cost documentation specifically recommends code-intelligence navigation to avoid unnecessary file reads and notes that an on-demand “codebase overview” skill can prevent the agent repeatedly reading multiple files just to reconstruct repository structure. [15]

The same principle applies across tools: `CODEMAP.md` should be **referenced only when the current agent actually needs architectural orientation**.

`tasks/NNN-....md`: this is the most important file per coding session

Example:

```
# TASK-007 — Create an opportunity

## Goal

An authenticated sales user can create an opportunity
for an existing company.

## Acceptance criteria

- User can open "New opportunity".
- Required fields: name, company, value, stage.
- Invalid values show validation errors.
- Successful submission persists the opportunity.
- User is redirected to the opportunity detail view.
- Refreshing the page shows the persisted data.

## In scope

- Opportunity database model if required
- Create-opportunity domain logic
- POST /opportunities
- New opportunity form
- Relevant unit/integration/E2E tests

## Out of scope

- Editing opportunities
- Deleting opportunities
- Pipeline drag-and-drop
- Forecast calculations
- Activity tracking

## Existing contracts / decisions

Read:
- PRODUCT.md sections: Opportunities
- docs/adr/0004-permissions.md
- docs/contracts/openapi.yaml
- docs/CODEMAP.md sections: API, Persistence, Web

## Relevant existing code

Start with:
- packages/domain/src/opportunities/
- apps/api/src/routes/opportunities/
- apps/web/src/features/opportunities/
- e2e/opportunities/

Inspect additional files only when dependencies require it.
```

```
## Constraints

- Preserve existing company API behaviour.
- No dependency upgrades.
- No unrelated refactors.
- Use existing form/validation patterns.

## Verification

- opportunity domain tests
- API integration test
- create-opportunity E2E test
- typecheck
- lint for touched packages

## Done

All acceptance criteria pass using the real application stack.
Update PROGRESS.md and stop.
```

This closely mirrors OpenAI's recommended phase files: purpose, relevant context, acceptance criteria and boundaries. [16]

`PROGRESS.md`: factual handoff, not diary

Do **not** dump entire session summaries into it.

Use:

```
# Project status

## Current baseline

Main commit: <sha>
Deployment state: staging healthy
Database migration head: <id>

## Completed

- TASK-001 Project foundation
- TASK-002 Authentication
- TASK-003 Company creation
- TASK-004 Company listing
- TASK-005 Contact creation
- TASK-006 Contact listing

## Verification at current baseline

- unit: PASS
- integration: PASS
```

```
- e2e critical path: PASS
- typecheck: PASS

## Known issues

- None blocking TASK-007.

## Next approved task

TASK-007 — Create an opportunity
```

OpenAI's workflow distinguishes progress/build-log evidence from phase context and specifically advises against using context files as transcripts, scratchpads or routine command-output archives. [14]

That separation is crucial for keeping token consumption stable as the project grows.

The complete workflow from idea to production

The following is the workflow I would actually use.

A **fresh session** below means a genuinely new Codex task/Claude Code context, not simply another prompt appended to the previous build conversation.

Stage	What the agent does	File/output created	When to start next session	What the next agent receives
Idea discovery	Researches domain, competitors/ technical constraints where needed; interviews you about users, jobs, workflows, edge cases, assumptions and non-goals. **No production code.**	Draft `PRODUCT.md`; optionally `docs/research/*.md` for genuinely durable research	When major product questions are resolved and you approve the intended MVP	`PRODUCT.md` only, plus any research source that materially affects decisions

Stage	What the agent does	File/output created	When to start next session	What the next agent receives
Requirements/ specification	Converts idea into testable behaviours, business rules, acceptance criteria and explicit exclusions. Removes ambiguity rather than choosing implementation details prematurely.	Final `PRODUCT.md`	**Fresh session after spec.** Anthropic explicitly recommends this for larger features/ specs. [6]	`PRODUCT.md`
Architecture and contracts	Chooses stack, major boundaries, auth/ permissions model, persistence approach, deployment architecture and initial cross-layer contracts. Does not implement features.	`ARCHITECTURE.md`, relevant `docs/adr/*.md`, initial DB/API contract, first `PLAN.md`	Fresh session once decisions are approved	`PRODUCT.md`, `ARCHITECTURE.md`, referenced ADRs/ contracts
Agent control plane	Creates the durable repository rules and lean handoff structure.	`AGENTS.md`, tiny `CLAUDE.md`, `PROGRESS.md`, `tasks/`, optionally `CODEMAP.md`	Continue into setup if this is a very small repo; otherwise fresh setup session	Agent instructions + architecture + `tasks/001-project-foundation.md`

Stage	What the agent does	File/output created	When to start next session	What the next agent receives
Project foundation	Creates repository skeleton, package structure, config, lint/typecheck/test infrastructure, CI, environment template, local DB/dev environment and minimal deployable shell. **No business features.**	Working skeleton, CI config, `.env.example`, deployment baseline, updated `CODEMAP.md`, `PROGRESS.md`	Fresh session after baseline checks pass and commit is clean	`AGENTS.md` automatically; TASK-001 result via `PROGRESS.md`; next task
First production slice	Performs targeted read-only exploration; implements schema/domain/API/UI/tests required for one user outcome; runs focused verification; checks diff; updates progress.	Application code + tests + any required migration/contract changes; `PROGRESS.md`	**Fresh session as soon as the slice is accepted/committed**	Next task file + progress + explicitly referenced contracts/ADRs
Every subsequent slice	Repeats the same bounded cycle. It does not "continue building the app generally".	One coherent product capability per change	Fresh session after every completed slice	Only current task sources

Stage	What the agent does	File/output created	When to start next session	What the next agent receives
Milestone integration	After several related slices, runs wider cross-feature/ E2E tests; fixes integration defects but adds no new feature scope.	Broader tests/fixes; release/milestone evidence	Fresh session after green milestone	Progress + milestone criteria + relevant failures only
Hardening	Reviews authz, migrations, failure modes, observability, security, accessibility, performance and recovery requirements according to your product's risks.	`RELEASE_CHECKLIST.md`, fixes/tests, operational docs	Fresh reviewer session after writer fixes	Current diff/release candidate + acceptance/security criteria
Independent review	A different/ fresh context examines the release candidate against requirements, architecture and diff, looking for correctness gaps and unintended scope changes.	Review findings; subsequent remediation task(s)	Writer receives only actionable findings; after fixes, review again if material	Diff/commit range + criteria, not original writer's reasoning

Stage	What the agent does	File/output created	When to start next session	What the next agent receives
Staging/ deployment preparation	Builds release artefact, runs migrations in staging, deploys, executes smoke/E2E checks, confirms logging/health/ rollback.	CI/CD config, `RUNBOOK.md`, deployment evidence	Fresh production deployment session after staging is accepted	Exact release SHA/ tag + production checklist + runbook
Production and verification	Deploys the already-reviewed release; verifies migrations, health, logs and critical user journey; avoids unrelated code changes.	Production release record, updated `PROGRESS.md` / release notes	Session ends when production verification passes	Future work starts with a new task/ session

OpenAI's recommended Codex harness follows the same philosophy: reusable prompts should point to repository files rather than repeating their contents; phases should state explicit scope and non-goals; implementation should be gated by verification; and each phase should stop when its assigned work is complete. [17]

What happens inside every vertical-slice session

A coding session should follow this exact loop:

```
START FRESH SESSION
      │
      ▼
Read AGENTS.md automatically
      │
      ▼
Read current TASK file
      │
      ▼
Read PROGRESS.md
      │
      ▼
Read only referenced ADR/contracts
      │
      ▼
```

```
Targeted read-only exploration
      │
      ▼
State 5–10 line implementation plan
      │
      ▼
Run narrow baseline test if useful
      │
      ▼
Implement smallest complete change
      │
      ▼
Focused tests
      │
      ▼
End-to-end acceptance test
      │
      ▼
Inspect full diff
      │
      ├── unintended changes → remove
      │
      ▼
Update PROGRESS.md
      │
      ▼
Commit/PR
      │
      ▼
STOP SESSION
```

Anthropic explicitly recommends giving the agent a verification signal such as tests/build output rather than relying on assertions that work is finished. [4] OpenAI likewise recommends phase-level verification and observed evidence rather than narrative claims of completion. [18]

Do not fully plan every slice on day one

Your initial `PLAN.md` should say roughly:

```
# Build plan

## Foundation
TASK-001 project foundation

## Identity
TASK-002 sign in
TASK-003 sign out
TASK-004 password reset

## Companies
```

```
TASK-005 create company
TASK-006 list companies
TASK-007 company details

## Contacts
...
```

Do **not** write a 2,000-line implementation plan for task 37 before task 1 exists.

Create the detailed `tasks/NNN.md` just before that work becomes actionable.

Why? Earlier implementation will change exact filenames, interfaces and discoveries. Prematurely detailed downstream plans become stale context that agents later have to reconcile.

OpenAI's workflow similarly recommends creating phase-specific context when a phase begins and retaining only discoveries materially important to future decisions. [14]

Session, context and anti-rewrite discipline

Start a fresh session much more often than most people do

For this workflow, the default boundary is:

> **new product slice = new agent session**

That is deliberately aggressive.

Anthropic says `/clear` should be used between unrelated tasks because irrelevant history can reduce performance, and that when Claude has been corrected more than twice on the same issue, a clean session with a better prompt is usually preferable to carrying accumulated failed approaches forward. [19]

Use the following policy:

Situation	Action
New feature/slice	**New session**
Feature specification finished, implementation about to begin	**New session**
Switching from implementation to independent review	**New session**
Architecture decision is complete; implementation begins	**New session**
Same unfinished feature, terminal was closed	Resume existing session
One failing test remains from current feature	Stay in same session
Long conversation but same coherent task	Compact, then continue

Situation	Action
Agent has misunderstood the same thing repeatedly	**Abandon/clear and restart with a better task prompt**
Starting release/security review	**Fresh reviewer context**
Production deployment	**Fresh, narrowly scoped deployment context**

Codex's `/compact` can condense a long session while preserving its critical details. [8] Claude supports `/compact`, `/clear`, resumable sessions and configurable compaction instructions. [13]

So the right distinction is:

Compaction is for extending one task. Fresh sessions are for changing tasks.

Do not put previous chat summaries into every new prompt

A common but expensive workflow is:

```
"Here is everything we have done so far..."
[8,000 token summary]
"Now implement feature 14."
```

Do not do this.

Use:

```
Implement tasks/014-invite-member.md.

Follow AGENTS.md.
Read PROGRESS.md and only the contracts/ADRs referenced by the task.
Begin with a targeted read-only preflight.
Do not start TASK-015.
```

That is the whole initial implementation prompt in many cases.

The repository documents carry the state.

Prevent agents from rewriting working code with four layers

Prompt instructions alone are not sufficient. Use overlapping controls.

Scope control. The task explicitly identifies in-scope work, non-goals and relevant files. OpenAI's Codex workflow recommends exactly this kind of explicit scope/boundary definition. [17]

Regression control. Require tests around existing behaviour before changing risky logic and make successful verification a completion condition.

Diff control. Every coding task ends with review of the entire Git diff and explicit removal of unrelated formatting, dependency upgrades or refactors. Codex has a `/diff` command specifically for viewing staged, unstaged and untracked changes. [8]

Git/checkpoint control. Commit every accepted slice. Claude's internal checkpoints are useful but Anthropic explicitly warns that they are not a replacement for Git. [20]

The wording I would repeatedly use is:

```
Preserve all currently passing behaviour not explicitly changed by this task.

Do not refactor code merely because you see a cleaner design.

Do not modify unrelated files.

Before completion, inspect the entire diff and remove any change that is not
necessary for the acceptance criteria.
```

Use fresh reviewers instead of asking the writer "are you sure?"

This is a surprisingly important optimisation.

Do not spend another twenty turns asking the implementation agent:

```
Check again.
Are you sure?
Review it again.
Any bugs?
```

It already carries the assumptions that produced the code.

Open a fresh review context and give it:

```
task specification
architecture constraints
git diff / commit range
tests
```

Anthropic explicitly recommends a writer/reviewer pattern and notes that a fresh context can improve review because the reviewer is not anchored to the reasoning that produced the implementation. [21]

A good review prompt is:

```
Review the diff for TASK-014 against tasks/014-invite-member.md.

Do not rewrite code.
```

```
Look only for:
1. unmet acceptance criteria,
2. correctness bugs,
3. missing edge-case tests,
4. security/permission failures,
5. breaking existing contracts,
6. unrelated scope changes.

Return findings ordered by severity with file/line references.
If there are no substantive findings, say so.
Do not suggest stylistic refactors.
```

This typically gives better signal with less conversational context.

Use subagents for discovery, not automatically for implementation

Subagents are useful but **not inherently token-efficient**.

OpenAI recommends them for moving noisy exploration, tests and log analysis out of the main thread so the parent receives summaries rather than raw intermediate material. [22] Claude similarly recommends investigation subagents because repository research may read many files and consume large amounts of main-session context. [20]

Good use:

```
Main implementation agent
    │
    ├── subagent: inspect existing auth pattern
    ├── subagent: find similar form implementation
    └── subagent: analyse failing E2E log
         ↓
      concise findings
```

Bad use:

```
8 agents simultaneously changing:
frontend
backend
schema
API
tests
types
deployment
documentation
```

This increases total inference and coordination. Anthropic explicitly notes that independent agent-team contexts increase total token usage roughly with team size. [23]

So, for a **token-minimised workflow**, default to **one writer agent**. Add read-only subagents when they save the parent from ingesting large volumes of repository material.

Parallel write sessions should use worktrees

Sometimes parallelism is worth paying for, particularly when tasks truly do not overlap.

Both Codex and Claude support Git-worktree-oriented workflows to isolate simultaneous changes; Claude specifically recommends worktrees so edits from independent sessions do not collide. [24]

Good parallel pair:

```
Worktree A → TASK-021 Stripe webhook handling
Worktree B → TASK-022 documentation site
```

Poor parallel pair:

```
Worktree A → opportunity API
Worktree B → opportunity database
Worktree C → opportunity frontend
```

The second group shares contracts and is better represented as one vertical slice.

Filter noisy output before the model consumes it

Tests and logs are often enormous context sinks.

Instead of sending:

```
20,000 lines of CI output
```

send:

```
failures
stack traces
surrounding relevant lines
test summary
```

Anthropic's current cost guidance specifically recommends hooks that pre-process large logs and demonstrates filtering test output to errors before Claude sees it. [15]

Likewise, use symbol/code intelligence rather than repeatedly reading entire directories. Anthropic notes that precise symbol navigation can replace multiple search-and-read operations. [23]

Prefer direct CLI operations when the alternative adds huge tool context

For Claude Code specifically, Anthropic says normal CLI tools such as `gh`, `aws`, `gcloud` and `sentry-cli` are generally more context-efficient than exposing a large number of MCP tool definitions, and recommends disabling unused MCP servers. [23]

This does not mean "never use MCP". It means:

```
Need one simple GitHub operation?
→ gh may be cheaper.

Need rich structured access across an external system?
→ MCP may be worth it.

Not using a server in this project?
→ don't keep unnecessary tools exposed.
```

What belongs in persistent context versus on-demand context

Use this hierarchy:

Information	Where it belongs	Automatically/routinely loaded?
Universal coding/ workflow rules	`AGENTS.md`	Yes
Claude bridge/special instructions	`CLAUDE.md`	Yes
Product truth	`PRODUCT.md`	Only product/planning work
Stable architecture	`ARCHITECTURE.md`	Architecture + relevant coding tasks
Expensive decisions	`docs/adr/*`	Only when referenced
API/schema contracts	`docs/contracts/*`	Only relevant slices
Repository orientation	`CODEMAP.md`	On demand
Ordered roadmap	`PLAN.md`	Planning work
Current work scope	`tasks/NNN.md`	Every implementation session
Current factual status	`PROGRESS.md`	Every implementation session
Session transcript	**Nowhere**	Never
Raw test logs	CI/build artefacts	Never unless debugging
Failed approaches	Usually nowhere	Preserve only if future agents genuinely need to avoid them

That last rule mirrors OpenAI's “materiality test”: retain information when omitting it would cause repeated discovery, change an interface/invariant or alter acceptance/verification—not merely because the agent happened to discuss it. [14]

Model routing and token economics

Model selection should be treated like compute allocation.

The wrong strategy is:

> “Use the smartest model at maximum reasoning on every step.”

OpenAI explicitly states that higher reasoning can improve complex tasks but also uses more tokens and recommends starting with the default/lower setting and increasing it only when required. Current Codex documentation says most tasks do not need Max or Ultra. [25]

Anthropic gives very similar guidance: Sonnet handles most coding tasks at lower cost than Opus; Opus should be reserved for complex architectural or multi-step reasoning, while Haiku fits simple subagent tasks. [23]

Current Codex routing

As of **11 September 2026**, OpenAI's Codex documentation lists Astra as its strongest option for hard end-to-end work, with GPT-5.6 Sol for complex/open-ended work, Terra as the everyday all-rounder and Luna as the fast/affordable choice for clearly specified repeatable work. Availability varies by account and rollout. [26]

For this workflow I would route work approximately as follows:

Work	Codex choice	Reasoning
Rename/update repetitive files	Luna	Low
Generate routine tests from explicit pattern	Luna/Terra	Low
Search/summarise known code areas	Terra	Low
Implement well-specified normal slice	Terra	Medium
Implement slice with meaningful domain logic	Terra/Sol	Medium
Diagnose difficult cross-layer bug	Sol	High
Architecture/trade-off analysis	Sol; Astra when genuinely hard	High
Difficult release/security reasoning	Sol/Astra	High
Massive ambiguous end-to-end problem	Astra	Only when justified

Work	Codex choice	Reasoning
Routine work	**Do not use Max/Ultra automatically**	—

This follows OpenAI's own positioning: Terra for everyday tool-using work, Luna for clear/repeatable tasks, Sol for ambiguous/high-value work and Astra for the hardest sustained workflows; the docs recommend using the lowest reasoning level that gets the result. [27]

An important detail: although Astra can now keep notes across context windows through an experimental context-management feature on some supported clients/accounts, OpenAI says the capability is currently rollout- and sign-in-dependent and off by default in the experiment. I would therefore **not design your project workflow around implicit model memory**. Repository memory remains the portable solution across Codex, Claude and future models. [27]

Current Claude Code routing

Anthropic's current aliases are deliberately role-oriented: `sonnet` points to the latest Sonnet for daily coding, `opus` to the latest Opus for complex reasoning, `haiku` to a fast efficient model for simpler tasks, and `fable` is intended for the hardest/longest-running tasks where available. [28]

Use:

Work	Claude model
Mechanical change	Haiku
Simple exploration subagent	Haiku
Normal implementation	Sonnet
Most debugging	Sonnet
Moderate code review	Sonnet
Complex architecture	Opus
Difficult systemic bug/security reasoning	Opus
Extremely demanding long-running problem	Fable where justified

Anthropic itself recommends **Sonnet for most coding, Opus for complex architectural/multi-step work and Haiku for simple subagents**. [23]

The escalation ladder

Do not choose the strongest model because a task *sounds important*. Escalate because a cheaper attempt has exposed reasoning complexity.

My preferred ladder is:

```
Explicit, mechanical task
      ↓
```

```
cheap/fast model
      │
      ├── succeeds → done
      │
      └── fails because task requires reasoning
                  ↓
            normal coding model
                  │
                  ├── succeeds → done
                  │
                  └── ambiguity/systemic complexity
                            ↓
                       strong model
                            │
                            └── highest tier only for
                                genuinely difficult cases
```

More concretely:

```
Codex:
Luna → Terra → Sol → Astra

Claude:
Haiku → Sonnet → Opus → Fable where appropriate
```

Do **not** rerun exactly the same weak prompt on a stronger model first.

Improve the information boundary:

```
What was ambiguous?
What contract was missing?
What failed?
What evidence exists?
What files are actually relevant?
```

Then escalate.

Where the expensive model produces the greatest return

Spend higher-quality reasoning on **high-leverage decisions**:

```
Product ambiguities
Architecture
Database invariants
Auth/security
Concurrency
Complex migrations
```

```
Public API design
Difficult systemic debugging
Release/security review
```

Do not spend it on:

```
Formatting
Boilerplate
Straightforward unit tests
Renaming
Simple CRUD once patterns exist
Updating documentation from known facts
Mechanical migrations
```

This gives you a useful asymmetry:

> **Use expensive intelligence to create good boundaries. Use cheaper intelligence to execute inside those boundaries.**

That principle is arguably more important than which vendor you choose.

A concrete operating protocol

Combining the research above, I would run an AI-built application under the following protocol.

The planning rule

Before coding, ensure the current task has:

```
one goal
explicit acceptance criteria
explicit non-goals
relevant context paths
contract references
verification commands/behaviour
```

Do not ask the coding agent to invent these while simultaneously implementing a large feature.

For an unusually large feature, Anthropic recommends first having Claude interview you and produce the specification, and then beginning implementation in a fresh session. [29]

The reading rule

Every writer receives the instruction:

```
Do not perform generic repository onboarding.
```

```
Read:
1. AGENTS.md / CLAUDE.md,
2. the current task,
3. PROGRESS.md,
4. only the architecture/contracts referenced by the task,
5. the directly relevant implementation/tests.

Expand discovery only when necessary to resolve an actual dependency or
unknown.
```

This is the mechanism that stops every new session from spending its first 30,000 tokens rediscovering the repository.

The editing rule

```
Before editing:
- inspect current implementation;
- identify an existing analogous pattern;
- check relevant tests.

During editing:
- smallest complete change;
- no opportunistic refactor;
- no dependency upgrades;
- no unrelated cleanup.

After editing:
- focused tests;
- acceptance verification;
- inspect complete diff;
- update progress;
- stop.
```

The commit rule

Use approximately:

```
one task
→ one bounded change
→ one testable outcome
→ one commit/PR
```

Not:

```
"Codex worked on the repository all afternoon"
→ 74 files changed
```

The handoff rule

At session completion, the coding agent does **not** write a giant prose summary.

It updates repository state:

```
code
tests
contract, if changed
ADR, if a new permanent decision occurred
PROGRESS.md
```

Its final response can be as small as:

```
TASK-007 complete.

Changed:
- ...
- ...

Verification:
- pnpm test opportunities: PASS
- pnpm e2e create-opportunity: PASS
- pnpm typecheck: PASS

PROGRESS.md updated.
No unresolved blockers.
```

Everything the next agent truly requires should already be in the repository.

The stop rule

This is extremely important:

```
Do not continue to the next feature merely because there is context
remaining.
```

Context remaining is not free future productivity. Continuing turns completed-feature context into baggage for the next feature.

Anthropic explicitly notes that stale context consumes tokens on every subsequent message and recommends clearing between tasks. [23]

The ideal completed slice often leaves unused context behind.

That is a feature, not waste.

Example prompts from zero to a production application

The following is a realistic sequence for creating a **B2B sales pipeline application** with organisations, contacts, opportunities and a pipeline.

The prompts are deliberately designed so you could use the sequence with either Codex or Claude Code. For the earliest conceptual stages, use a stronger reasoning model. Once the specification and architecture stabilise, most implementation should move to the normal/cheaper coding tier. This reflects the current model-routing recommendations from OpenAI and Anthropic. [30]

Session: Product discovery — strong model

```
I want to build a B2B sales pipeline web application for small sales teams.

Do not write code.

Your job is product discovery.

Interview me about:
- target users,
- their existing workflow,
- the core job the product solves,
- organisations and contacts,
- opportunities/pipeline,
- permissions,
- collaboration,
- search/filtering,
- notifications,
- reporting,
- integrations,
- onboarding,
- billing,
- security/privacy,
- MVP versus post-MVP,
- edge cases and failure conditions.

Focus on decisions that materially affect the product.
Do not ask questions whose answer can safely be deferred.

After the interview, create PRODUCT.md containing:
- problem statement,
- target users,
- primary workflows,
- functional requirements,
- business rules,
- MVP scope,
- explicit non-goals,
- acceptance-level behaviours,
- unresolved decisions.
```

```
Do not choose the technical architecture yet.
Do not implement anything.
```

Start a fresh session after `PRODUCT.md` is approved.

Anthropic expressly recommends fresh execution context after a larger specification has been completed. [6]

Session: Architecture — strong model

```
Read PRODUCT.md.

Do not write application code.

Design the minimum production architecture required to implement the MVP.

Optimise for:
- simplicity,
- low operational burden,
- clear module ownership,
- type safety,
- easy automated testing,
- easy deployment,
- small AI-agent task boundaries.

Determine:
- frontend architecture,
- backend architecture,
- database,
- authentication,
- authorisation,
- API approach,
- validation,
- background jobs only if actually required,
- testing layers,
- observability,
- deployment,
- secrets/configuration,
- migration strategy.

Create:

ARCHITECTURE.md
docs/adr/0001-stack.md
docs/adr/0002-authentication.md
docs/adr/0003-authorization.md
docs/contracts/openapi.yaml where appropriate
PLAN.md

PLAN.md should contain ordered vertical user-facing slices.
```

```
Do not create detailed implementation instructions for all future slices.
Do not write feature code.

Call out assumptions and irreversible/high-cost decisions.
Prefer boring, proven infrastructure unless PRODUCT.md requires otherwise.
```

Start a fresh session.

Session: Agent control plane and project foundation task

```
Read PRODUCT.md, ARCHITECTURE.md, PLAN.md and accepted ADRs.

Set up the repository's AI-agent working system.

Create:

AGENTS.md
CLAUDE.md
PROGRESS.md
docs/CODEMAP.md
tasks/001-project-foundation.md

Requirements:

AGENTS.md must be concise and contain:
- canonical source-of-truth locations,
- read-only preflight requirement,
- scope discipline,
- no unrelated refactors,
- no dependency upgrades unless explicitly required,
- preserve working public behaviour,
- focused verification,
- full diff review before completion,
- update PROGRESS.md,
- stop after the current task.

CLAUDE.md should import @AGENTS.md rather than duplicate it.

PROGRESS.md should contain factual current state only.

CODEMAP.md should document major modules/entry points, not every file.

TASK-001 should define the minimum project foundation needed before the
first user-facing vertical slice.

Do not implement the application yet.
```

The `CLAUDE.md` import pattern is explicitly supported by current Claude Code documentation and avoids maintaining duplicate persistent instructions. [12]

Session: Foundation — normal coding model

```
Implement tasks/001-project-foundation.md.

Follow AGENTS.md.

Read:
- PROGRESS.md
- ARCHITECTURE.md
- only the ADRs referenced by TASK-001

Perform a read-only preflight before editing.

Create only the minimum deployable/testable project foundation required
by the task.

Do not implement sales-domain features.

Before completion:
- run all TASK-001 verification,
- inspect the complete git diff,
- remove unrelated changes,
- update docs/CODEMAP.md if structure changed,
- update PROGRESS.md with observed results,
- stop.

Do not start TASK-002.
```

Fresh session.

Session: Create the first slice task

This planning operation is inexpensive and can generally use the normal model:

```
Read PRODUCT.md, PLAN.md, PROGRESS.md and ARCHITECTURE.md.

The next roadmap item is authentication.

Create tasks/002-sign-in.md.

It must define exactly one vertical outcome:
an existing user can sign in and reach the authenticated application.

Include:
- acceptance criteria,
- in scope,
- explicit non-goals,
- relevant ADR/contracts,
- likely relevant code areas,
```

```
- constraints,
- verification,
- definition of done.

Do not implement it.

Keep the task self-contained but reference canonical files instead of
copying their contents.

Do not create TASK-003 yet.
```

OpenAI specifically recommends prompts referencing existing goal/plan files rather than copying their content and requiring explicit immediate scope/non-goals. [5]

Fresh implementation session: Authentication slice

```
Implement tasks/002-sign-in.md.

Follow AGENTS.md.

Start by reading:
- tasks/002-sign-in.md
- PROGRESS.md
- only the ADRs/contracts named by the task
- directly relevant existing code/tests

Do not scan the whole repository unless the task cannot be completed
without additional discovery.

Before editing, give me a concise plan based on what actually exists.

Implement the smallest complete end-to-end solution.

Do not:
- add registration,
- add password reset,
- refactor unrelated authentication infrastructure,
- upgrade dependencies,
unless TASK-002 explicitly requires it.

Run focused tests plus the end-to-end acceptance check.

Then:
- inspect the full diff,
- remove unintended changes,
- update PROGRESS.md,
- stop.

Do not begin another task.
```

Fresh session: Company creation slice

After creating `tasks/003-create-company.md`:

```
Implement tasks/003-create-company.md.

Follow AGENTS.md.

Use the existing authentication and application patterns.

Read only:
- current task,
- PROGRESS.md,
- contracts/ADRs referenced by the task,
- relevant company/domain/API/UI/test code.

Implement the complete vertical path:

database change if required
→ domain operation
→ API
→ real frontend integration
→ automated verification

Do not implement editing, deletion, import, tagging, search or filtering
unless they are explicit acceptance criteria.

Preserve all existing passing behaviour.

Inspect the complete diff and update PROGRESS.md before stopping.
```

Fresh session: Company listing

```
Implement tasks/004-list-companies.md.

Follow AGENTS.md.

This is a new task: do not rely on conversation history from TASK-003.

Begin from the current repository state and task documentation.

Use existing company contracts and patterns.
Do not redesign the company module.

Complete only this slice, verify it end-to-end, update PROGRESS.md and stop.
```

Notice the pattern: **the prompts become shorter as the repository gets better**.

That is exactly what you want.

By slice 20, your typical prompt should not be 2,000 words. It should look like:

```
Implement tasks/020-move-opportunity-stage.md.

Follow AGENTS.md.
Read PROGRESS.md and the sources referenced by the task.
Targeted preflight only.
Verify all acceptance criteria.
Inspect the full diff.
Update PROGRESS.md and stop.
```

A mature repository should carry almost all repeated context itself.

Session: Difficult bug — stronger model only when needed

Suppose a race condition appears in opportunity updates:

```
Investigate the concurrency failure described in
tasks/bug-031-opportunity-stage-race.md.

Do not edit initially.

Read only the task plus relevant opportunity persistence/service code and
tests.

Reproduce the problem if possible.

Determine the root cause, distinguishing evidence from hypotheses.

Before making changes, report:
1. reproduction,
2. root cause,
3. invariant that is being violated,
4. smallest safe correction,
5. regression test required.

Then implement only if the evidence supports the plan.

Do not redesign the opportunity architecture unless the current architecture
cannot satisfy the required invariant.

Run the regression test and relevant integration tests.
Inspect the full diff.
Update PROGRESS.md.
```

This is where Sol/Astra or Opus may justify their additional reasoning cost rather than using them for every CRUD slice. Current OpenAI and Anthropic guidance both recommend escalating stronger reasoning/models for more difficult, architectural or multi-step work instead of making them the default for simple work. [30]

Fresh session: Milestone integration

```
This is an integration-verification task, not a feature-development task.

Read:
- PRODUCT.md acceptance criteria for the completed milestone,
- PROGRESS.md,
- completed task files for Companies, Contacts and Opportunities,
- relevant E2E tests.

Do not add new product functionality.

Run the complete milestone user journey:

sign in
→ create company
→ create contact
→ create opportunity
→ change opportunity stage
→ reload
→ verify persisted state

Also run the milestone integration suite.

For every failure:
- determine whether it is a product defect, test defect or environment issue;
- make the smallest necessary fix;
- do not opportunistically refactor.

Update PROGRESS.md with observed verification evidence and stop.
```

Fresh session: Independent release reviewer

Use a strong reviewer, but do not give it the writer's entire conversation:

```
Act as an independent release reviewer.

Do not modify code.

Review the release candidate from <base-commit> to <candidate-commit>.

Read:
- PRODUCT.md,
- ARCHITECTURE.md,
```

```
- relevant ADRs,
- RELEASE_CHECKLIST.md,
- the git diff,
- relevant tests.

Check only for substantive release risks:

- unmet product requirements,
- correctness bugs,
- authentication/authorisation flaws,
- data-integrity risks,
- unsafe migrations,
- API contract regressions,
- missing critical tests,
- failure/recovery problems,
- accidental out-of-scope modifications.

Do not report stylistic preferences or optional refactors.

For each finding provide:
severity
file/location
requirement/invariant violated
why it matters
smallest recommended remediation

If no substantive issue exists, explicitly say so.
```

Anthropic specifically recommends independent fresh-context review and notes that such a reviewer can evaluate the result without inheriting the writer's assumptions. [21]

Fresh remediation session

```
Read RELEASE-REVIEW.md.

Convert only accepted findings into a bounded remediation task.

Implement those fixes.

Do not address rejected suggestions or unrelated quality improvements.

Run the verification associated with each finding plus the release-critical
suite.

Inspect the final diff.
Update PROGRESS.md.
Stop.
```

Fresh session: Staging

```
Prepare and verify the current release candidate in staging.

Do not add features and do not refactor application code.

Read:
- PROGRESS.md
- RELEASE_CHECKLIST.md
- deployment configuration
- RUNBOOK.md if present

Use the exact reviewed release commit.

Verify:
- build,
- configuration/secrets presence without exposing values,
- migrations,
- deployment health,
- database connectivity,
- critical smoke tests,
- authentication,
- primary end-to-end journey,
- logging/monitoring,
- rollback procedure.

Create/update RUNBOOK.md with only operationally necessary information.

Record observed results in the release checklist.

Stop after staging verification.
Do not deploy production as a side effect of this task.
```

Fresh session: Production

```
This session is exclusively for deploying the already-approved release.

Release commit/tag: <exact identifier>

Read:
- RELEASE_CHECKLIST.md
- RUNBOOK.md
- PROGRESS.md

Do not edit application code.

Before deployment:
- confirm the release identifier,
- confirm required migrations,
- confirm production configuration,
- confirm rollback target/procedure.
```

```
Deploy only the approved artefact.

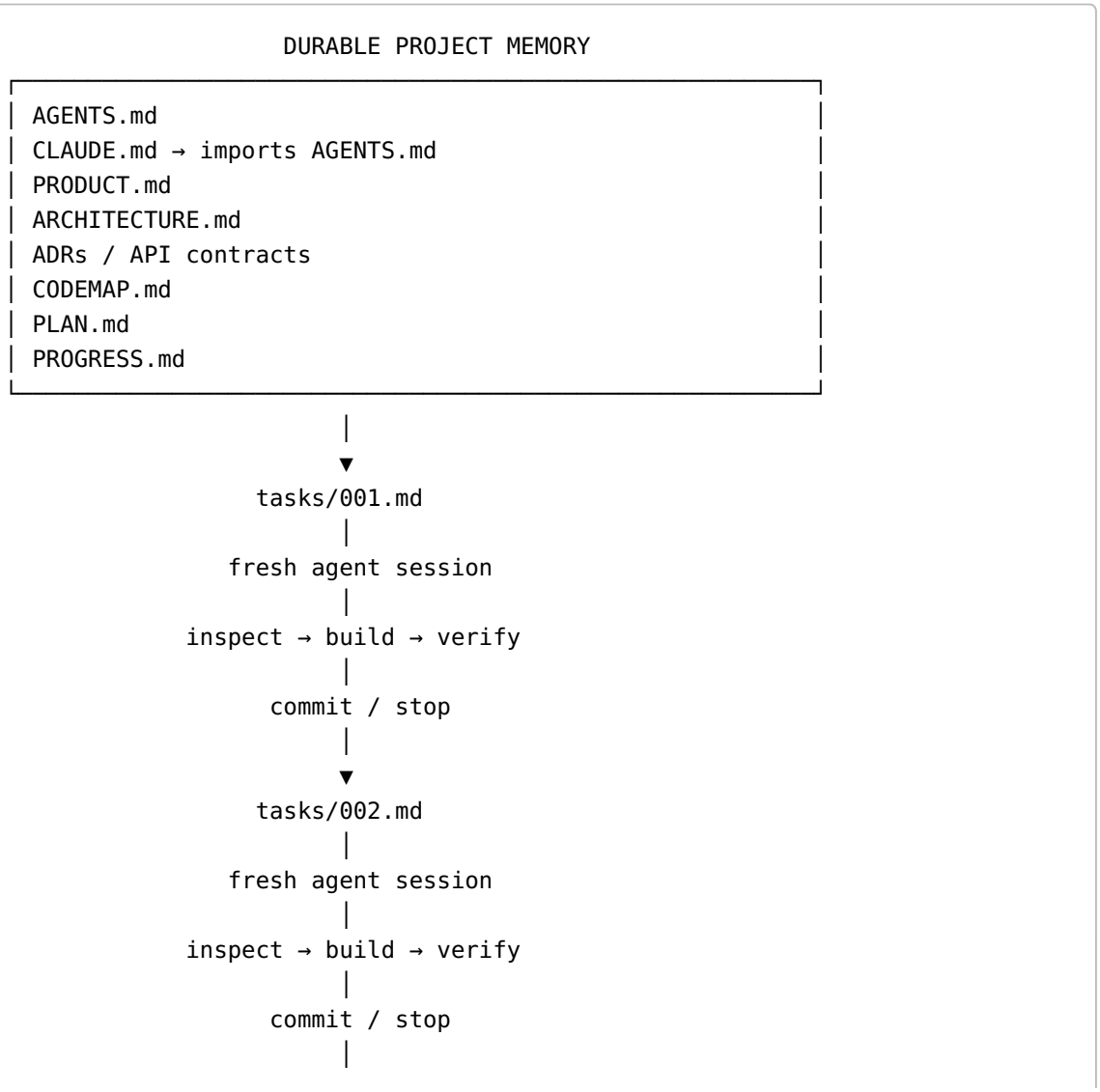
After deployment verify:
- deployment health,
- migration completion,
- authentication,
- critical production smoke journey,
- error/health signals.

If verification fails, follow the documented rollback procedure rather than
attempting unrelated live code changes.

Record the deployed version and verification evidence in PROGRESS.md /
release records.

Stop.
```

The final system therefore looks like this:



```
                ▼
               ...
                │
                ▼
        milestone verification
                │
                ▼
        fresh independent review
                │
                ▼
              staging
                │
                ▼
            production
```

The central optimisation is not a clever prompt or a larger context window. It is **making each agent disposable**.

A well-run project should reach the point where an implementation agent needs only four things:

```
repository instructions
+
one task specification
+
current factual progress
+
the small set of code/contracts relevant to that task
```

Everything else should be either discoverable on demand or unnecessary.

That simultaneously addresses all eight of the objectives in the question: context stays small because sessions are bounded; repository analysis is not repeated because stable architecture and code maps exist; working code is protected by scope, tests and diff review; cross-session knowledge lives in version-controlled artefacts; `AGENTS.md` / `CLAUDE.md` stay concise; new sessions begin at feature boundaries; the application is decomposed into sequential independently verifiable slices; and expensive reasoning is reserved for the comparatively small fraction of work where it has real leverage. Those principles are directly consistent with the current Codex workflow guidance on scoped phase files and material handoff context, and Claude Code's guidance on aggressive context management, fresh execution contexts, focused persistent instructions and model-tier selection. [31]

---

[1] [5] [9] [14] [16] [17] [18] [31] Iterating Development Workflows with Codex
https://developers.openai.com/cookbook/examples/codex/iterating-development-workflows-with-codex

[2] [4] [6] [7] [11] [19] [20] [21] [29] Best practices for Claude Code - Claude Code Docs
https://code.claude.com/docs/en/best-practices

[3] [13] [15] [23] Manage costs effectively - Claude Code Docs
https://code.claude.com/docs/en/costs

8 Developer commands | ChatGPT Learn
https://developers.openai.com/codex/cli/reference

10 Custom instructions with AGENTS.md | ChatGPT Learn
https://developers.openai.com/codex/guides/agents-md

12 How Claude remembers your project - Claude Code Docs
https://code.claude.com/docs/en/memory

22 Subagents | ChatGPT Learn
https://learn.chatgpt.com/docs/agent-configuration/subagents

24 Worktrees | ChatGPT Learn
https://learn.chatgpt.com/docs/environments/git-worktrees

25 26 27 30 Models | ChatGPT Learn
https://developers.openai.com/codex/models

28 Model configuration - Claude Code Docs
https://code.claude.com/docs/en/model-config